

Schema Reconciliation — Database Design vs. Live Implementation

Project: ICT302 Secure QR Code Management System **Date:** 15 July 2026 **Purpose:** Align the proposed database schema with the implemented schema currently live in Supabase, before frontend integration begins (schema changes cascade into API contracts, so this must be settled first).

Summary

The two designs agree on the fundamentals: five tables, status-driven QR lifecycle, a dedicated activity log, and scan telemetry separated from incident reports. This document covers the remaining differences, why each was resolved the way it was, and one proposal from the design schema that has been adopted into the live system.

The guiding principle: the backend is fully built, deployed to Render, and verified end-to-end (including a physical phone-scan test on the live domain). Where the two schemas differ, we keep the implemented version unless there is a functional reason to change it — and we adopt design-side ideas where they add value without breaking verified behaviour.

Adopted from the design schema

`QR_Code.user_id` → adopted as `QrCode.createdById` (optional)

Tracking who generated each QR code is a good audit feature, especially once multiple admins exist. It has been added to the live schema as an optional field (`createdById String?`) with a relation to (`User`). It is optional rather than NOT NULL so that QR codes generated before this change remain valid.

`User.lastLogin` → adopted (optional)

Discovered as a direct-to-database addition on the live User table during reconciliation. Adopted into the canonical schema: it costs a single update statement in the login endpoint and gives admins useful account-activity visibility on the Users page. Recorded here so the schema file, the database, and the documentation stay in agreement.

Not adopted — functional conflicts

These three items conflict with client requirements or with decisions already validated in the deployed system. They are not style preferences.

1. `QR_Scan_Log.qr_id NOT NULL`

The live schema keeps `ScanLog.qrCodeId` optional.

The system's core purpose is tamper detection. When someone physically replaces a QR code, the scanned token does not exist in our database — there is no parent QR code to reference. If `qr_id` is NOT NULL, the database physically cannot record the most important scan events in the system: the tampered ones. The optional foreign key is deliberate, and the verify endpoint already logs every scan attempt (including invalid/unknown tokens) under this design.

2. `QR_Incident_Report.photo` bytea NOT NULL

The live schema stores a **photo URL** (`Alert.photoUrl`), with the image file itself in Supabase Storage (`alert-photos` bucket).

Storing binary image data in the database bloats it quickly and makes backups and queries slower. The implemented upload pipeline (multer memoryStorage → Supabase Storage → URL saved to DB) is complete and tested, and follows the platform constraint that Render's filesystem is ephemeral. Additionally, NOT NULL would reject reports submitted without a photo, which should remain possible.

3. `QR_Incident_Report.user_id` NOT NULL

The live schema has **no user foreign key on alerts**; reporter name and contact are optional free-text fields.

This comes directly from the client brief, which specifies reporter name and contact information are included only *"if user wishes to share."* Anonymous reporting is a requirement. Under the system's access model, regular scanner users (Apple/Google camera) have no account at all, so requiring a logged-in `user_id` would make it impossible for the primary reporting audience to report.

Kept as implemented — with rationale

Item	Design schema	Live schema	Rationale
Status values	<code>'Active'</code> , <code>'Blacklisted'</code> (capitalised)	<code>'active'</code> , <code>'blacklisted'</code> (lowercase)	All backend logic, seed data, and API responses already use lowercase. Changing casing would touch every endpoint for zero functional gain.
User roles	<code>'Administrator'</code> / <code>'User'</code>	<code>'admin'</code> / <code>'user'</code>	Same values conceptually; middleware (<code>requireAdmin</code>), seed script, and JWT payloads already use the lowercase forms.

Item	Design schema	Live schema	Rationale
GPS storage	<code>location varchar</code> (single field)	<code>gpsLat</code> / <code>gpsLng</code> (two Float fields)	Numeric coordinates are required for map display and distance calculations on the dashboard. A single string cannot be queried or plotted.
Alert status stages	<code>Pending</code> / <code>Under Investigation</code> / <code>Resolved</code>	<code>new</code> / <code>resolved</code>	The three-stage flow is a good idea but requires frontend and backend changes on both alert pages. Deferred as a future enhancement (will be noted in the final report) given the 1 August deadline.
Naming convention	<code>snake_case</code> (<code>created_date_time</code>)	<code>camelCase</code> (<code>createdAt</code>)	Prisma/JavaScript convention; the API contracts and frontend already consume camelCase field names.

Fields in the live schema not present in the design schema

These support features that are already built and verified:

- `QrCode.label` — human-readable name shown in the dashboard table and details view
- `User.phoneNumber`, `User.organisationId` — registration form fields (phone optional)
- `User.resetToken`, `User.resetTokenExpiresAt` — forgot-password flow
- `ScanLog.userAgent` — drives the app-vs-browser branching (`SecureQRApp/1.0` detection), which is the backbone of the backward-compatibility requirement
- `ScanLog.gpsLat` / `gpsLng` — optional scan location telemetry
- `ActivityLog.qrCodeId` **optional** — allows system-level events not tied to a specific QR code

Fields removed during this reconciliation

As part of settling the canonical schema, the following unused columns were removed from the live schema (verified via codebase audit — no code reads or writes them):

- `User.organisationId` — a remnant of the early multi-organisation design in the PMP; no Organisation table exists, the registration form has no such field, and no endpoint

populates it. It was always null.

- `ActivityLog.ipAddress` — never written by any of the three ActivityLog write sites (QR status change, alert created, alert resolved/reopened). IP telemetry is the responsibility of `ScanLog`, which retains its `ipAddress` field per the client's telemetry requirements; `ActivityLog` is the curated event feed and does not need it.
- `User.status` (`active`/`disabled`) — found as a direct-to-database addition on the live User table. Not adopted because an account-status column is a *functional promise*: honouring it requires login rejection, auth-middleware checks for already-issued JWTs, deactivation UI, and an extension of the last-admin guard. Leaving the column without that enforcement would be worse than not having it — an admin could mark an account "disabled" while the system continues to accept its logins. Account deactivation is recorded as a **future enhancement** (alongside the three-stage alert status), consistent with the current scope decision of "role change only, no user deletion."

Process note: `User.lastLogin` and `User.status` were added directly in Supabase rather than through the Prisma schema, and were only discovered when `prisma db push` flagged them. Going forward, all schema changes should be made through `prisma/schema.prisma` in the repository (via Roy) so the schema file, the database, and this document cannot silently diverge — per the team's change-management process, which freezes database design changes once coding has begun.

Data type mapping — Design Document vs. live implementation 📁

The Architecture Design Document specifies column types such as `uuid`, `character_varying`, and DB-level `CHECK` constraints. The live implementation uses Prisma ORM's standard PostgreSQL mappings, which differ in physical representation. **These are physical-level differences only — the logical design intent (unique identifiers, string fields, restricted value sets) is fully preserved.** This is a normal logical-design vs. physical-implementation distinction, not a functional deviation.

Equivalence table

Design Document	Live implementation (Prisma)	Equivalence rationale
<code>uuid</code> primary/foreign keys	<code>text</code> columns containing UUID strings (<code>@default(uuid())</code>)	Same UUID values, same uniqueness guarantees. Native <code>uuid</code> saves marginal storage but converting now would require type-change migrations across every PK/FK on a verified, deployed backend — high risk, no functional gain.

Design Document	Live implementation (Prisma)	Equivalence rationale
<code>character varying</code>	<code>text</code>	In PostgreSQL, unbounded <code>varchar</code> and <code>text</code> are identical in storage and performance; the PostgreSQL documentation treats them as interchangeable.
<code>timestamp</code>	<code>timestamp(3)</code>	Same semantics; Prisma defaults to millisecond precision.
DB-level <code>CHECK</code> constraints on status/role values	Application-level validation in route handlers and services	Both enforce restricted value sets. The implemented approach validates at the API boundary, which is where all writes occur (no direct DB access path exists). All endpoints enforcing these values are built and verified. Note the accepted value casing follows the implementation (<code>active</code> , <code>admin</code>) — lowercase), as documented in the "Kept as implemented" section above.

Action items

- Design Document revision (owner: Alvin, technical input: Roy)** — add an "Implementation Notes" subsection recording the physical type mappings above, or record them in the Final Report's Design Deviations table alongside the existing WBS 5.1.2 amendment ("Continue to destination" button → app-required model). Both deviations follow the same precedent: implementation decisions documented with rationale rather than reverted.
- System testing alignment (owner: Vanessa, before Sprint 5 starts 18 July)** — test cases must be written against the **live schema** (this document + `prisma/schema.prisma`), not the Design Document's type notation. In particular: status and role values are lowercase (`active`, `suspicious`, `blacklisted`, `expired`; `user`, `admin`), alert statuses are `new`/`resolved`, and IDs are UUID strings. Writing test expectations from the Design Document's capitalised values would produce false failures.
- Client-facing note (owner: Patricia, if required)** — per the change-management process, the database design is frozen once coding begins; this reconciliation confirms and documents the as-built design rather than changing it. No functional behaviour changes, so this is expected to fall under documentation alignment rather than a client-approval change.

Data dictionary — canonical schema

Every table's purpose and the reason each column exists. This is the reference for test-case writing (Vanessa), Design Document revision (Alvin), and the Final Report appendix.

User — authentication accounts

Holds every registered account: admins who manage the platform and app users who register through the mobile app. Regular scanner users (Apple/Google camera) have **no row here by design** — public QR verification is account-free.

Column	Type	Why it exists
<code>id</code>	text (UUID), PK	Stable unique identifier; referenced by <code>QrCode.createdById</code> .
<code>email</code>	text, unique	Login identifier; uniqueness enforced at DB level to prevent duplicate accounts.
<code>passwordHash</code>	text?	bcrypt hash — plain-text passwords are never stored (security requirement).
<code>fullName</code>	text?	Registration form field; shown on the Users page and Settings profile.
<code>phoneNumber</code>	text?	Optional registration field; editable in Settings.
<code>role</code>	text, default <code>user</code>	Drives authorisation: <code>requireAdmin</code> middleware gates the dashboard and admin API. Every new registration is <code>user</code> ; promotion is admin-only, with a last-admin demotion guard.
<code>resetToken</code>	text?	Single-use forgot-password token; null except during an active reset.
<code>resetTokenExpiresAt</code>	timestamp?	Expiry for the reset token, limiting the window of validity.
<code>lastLogin</code>	timestamp?	Updated on each successful login; gives admins account-activity visibility on the Users page.
<code>createdAt</code>	timestamp	Account registration time, for audit and display.

QrCode — the managed asset

One row per generated QR code. The printed QR encodes only a URL containing the signed token; this row is the server-side source of truth about that code's identity, destination, and lifecycle state.

Column	Type	Why it exists
<code>id</code>	text (UUID), PK	Identifier used across scan logs, alerts, activity entries, and admin detail views.
<code>token</code>	text, unique	The signed QR-JWT embedded in the printed code. Unique so a scanned token resolves to exactly one row; the QR PNG is regenerated from it on demand (images are never stored).
<code>label</code>	text?	Human-readable name ("Cafe front door") for the dashboard table and details view — admins can't recognise codes by UUID.
<code>destinationUrl</code>	text	Where a verified scan ultimately leads (app path only, per the app-required access model).
<code>status</code>	text, default <code>active</code>	The single overwritten lifecycle state (<code>active</code> / <code>suspicious</code> / <code>blacklisted</code> / <code>expired</code>) that drives verify-time behaviour. History lives in the logs, not here.
<code>expiresAt</code>	timestamp?	Configurable expiry (client requirement: "limited expiry timeout"); verify rejects past-expiry tokens.
<code>createdAt</code>	timestamp	Generation time, for display and metrics.
<code>createdById</code>	text? → User	Audit trail: which admin generated the code. Optional so pre-existing rows remain valid. (Adopted from the design schema.)

ScanLog — raw scan telemetry

Append-only record of **every** scan attempt, successful or not. This is the analytics dataset behind the dashboard's scan metrics and the client's telemetry requirement (timestamp, device type, IP address, validation result).

Column	Type	Why it exists
<code>id</code>	text (UUID), PK	Row identifier.
<code>qrCodeId</code>	text? → QrCode	Deliberately optional: a tampered or unknown token has no parent QR code, and those scans are precisely the ones a tamper-detection system must record.
<code>scannedAt</code>	timestamp	When the scan happened; powers the weekly scan-volume chart.
<code>ipAddress</code>	text?	The only location signal available for browser scans (no GPS on a GET request); raw material for detecting geographic anomalies such as QR replication (risk R04).
<code>userAgent</code>	text?	Records which client scanned — <code>SecureQRApp/1.0</code> vs. browser — the backbone of the app/browser branching and backward-compatibility verification.
<code>gpsLat</code> , <code>gpsLng</code>	float?	Precise scan location when the mobile app provides it (per WBS 4.5.5, "GPS if provided"). Null for browser scans by nature.
<code>result</code>	text	Validation outcome (<code>valid</code>)/(<code>expired</code>)/(<code>invalid</code>)/(<code>blacklisted</code>) — the dimension every scan metric slices on.

`Alert` — tamper incident reports

One row per tamper report submitted from the mobile app. Field set mirrors the client brief's Expected Outcomes for alerts.

Column	Type	Why it exists
<code>id</code>	text (UUID), PK	Row identifier.
<code>qrCodeId</code>	text → QrCode	The code being reported; report submission also flips that code's status to <code>suspicious</code> .
<code>reporterName</code>	text?	Optional per the brief ("if user wishes to share") — anonymous reporting is a requirement.
<code>contactInfo</code>	text?	Optional for the same reason.
<code>gpsLat</code> , <code>gpsLng</code>	float?	"Alert should contain Location (using mobile device information)" — numeric pair so the dashboard can plot and measure it.
<code>photoUrl</code>	text?	URL of the photo in Supabase Storage (<code>alert-photos</code> bucket). URL only, never binary — keeps the DB lean and survives Render's ephemeral filesystem.
<code>description</code>	text?	"Description of device or location" from the brief.
<code>status</code>	text, default <code>new</code>	Admin triage state (<code>new</code> / <code>resolved</code>); drives the Alerts-page badge counts. Three-stage flow deferred as future enhancement.
<code>createdAt</code>	timestamp	Report submission time.

`ActivityLog` — curated dashboard event feed

Human-readable record of *meaningful* system events, feeding the Recent Activity feed. Deliberately separate from `ScanLog`: `ScanLog` is high-volume raw telemetry for analytics; `ActivityLog` is low-volume curated narrative for humans. **Never merge them.**

Column	Type	Why it exists
<code>id</code>	text (UUID), PK	Row identifier.
<code>qrCodeId</code>	text?	The related code, when there is one; optional so system-level events can also be logged.
<code>type</code>	text	Machine-readable event category (<code>status_changed</code> , <code>alert_created</code> , <code>alert_resolved</code> , <code>alert_reopened</code>) for filtering and icons.
<code>message</code>	text	The human-readable sentence displayed in the feed.
<code>status</code>	text?	Contextual state attached to the event (e.g. the new QR status, or <code>new</code> / <code>resolved</code>), rendered as a badge in the feed.
<code>createdAt</code>	timestamp	Event time, for feed ordering.

`(_prisma_migrations)` also exists in the database: it is Prisma's internal migration ledger, not part of the application schema. It must not be dropped and does not appear in ERDs.)

Resulting canonical schema

The canonical schema is the live Prisma schema in the repository (`prisma/schema.prisma`) on `RoyYoo716/ICT302`), now including `QrCode.createdById`. Any future schema change should go through the team's change-management process before implementation, since the schema is now the contract the frontend will be built against.